

Estrutura de Dados II

Estrutura de dados Hierárquicas: **Árvores**



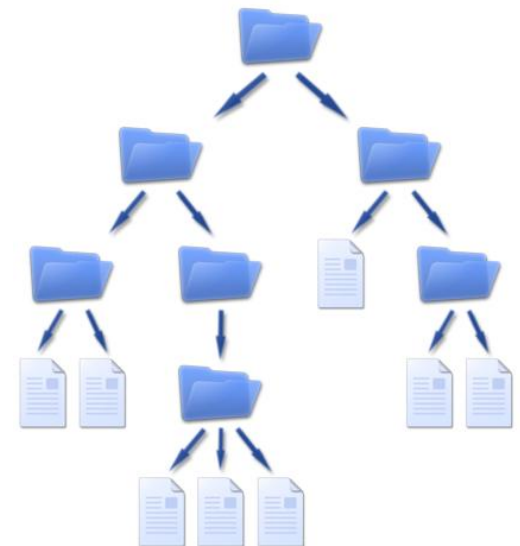
Árvores: Motivação

Em computação, há aplicações que necessitam de estruturas mais complexas que as estudadas até agora (lista, fila, pilha)

Um problema comum é como visualizar o conjunto de diretórios utilizando listas, pilhas e filas?

Estruturas de Árvores solucionam esse problema e além disso podem ser usados para modelar outros:

- Sistemas de arquivos
- Compressão de dados
- Busca eficiente
- Banco de dados
- Ordenação



Árvores: Motivação

Existem algoritmos eficientes para o tratamento de árvores.



Algumas árvores podem definir uma ordenação implícita, o que facilita a execução do algoritmo, com um tratamento condicional simples.

Árvores: Definição

Árvores são estruturas de dados que se caracterizam por uma organização hierárquica (**relação hierárquica**) entre seus elementos.



Essa organização permite a definição de algoritmos relativamente **simples**, **recursivos** e de **eficiência bastante razoável**.

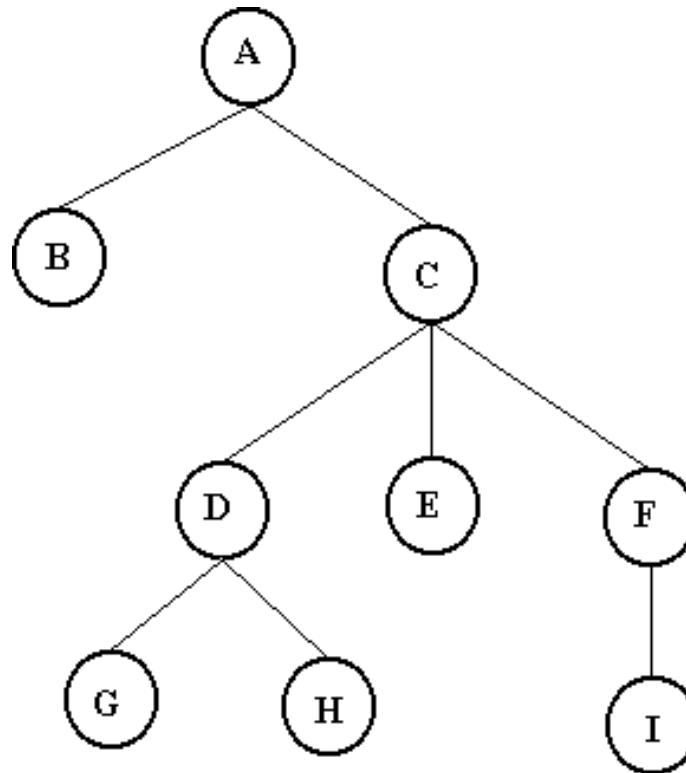
Árvores: Representação

Uma estrutura de dados do tipo árvore pode ser representada de forma:

- Hierárquica
- Expressão *parentetizada* (parênteses aninhados)
- Diagramas

Árvores: Representação

- Representação Hierárquica

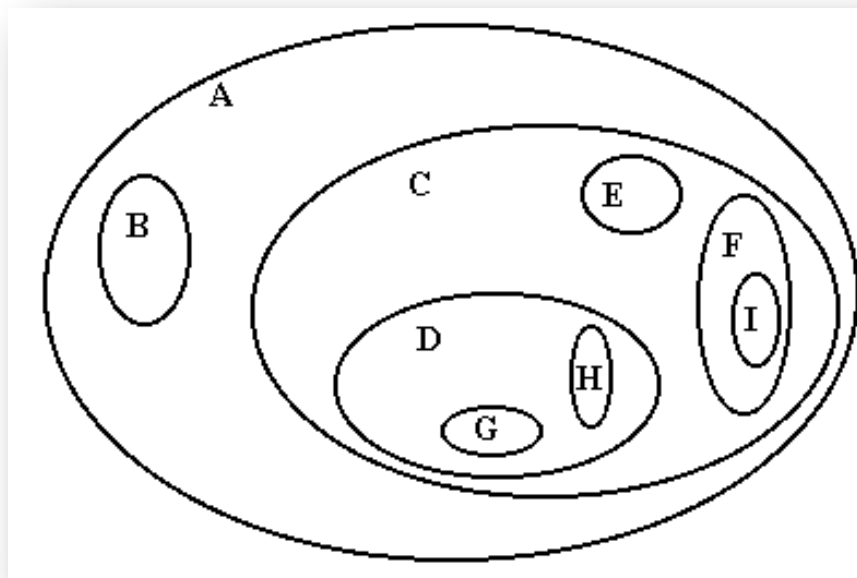


Árvores: Representação

- Representação em Parênteses aninhados

(A (B) (C (D (G) (H)) (E) (F (I)))))

- Representação em Diagramas de Inclusão



A é raiz
B e C filhos de A, D,
E e F filhos de C

Árvores: Definição

Uma árvore **N** é um conjunto finito de elementos denominados **NÓS** ou **VÉRTICES** tais que:

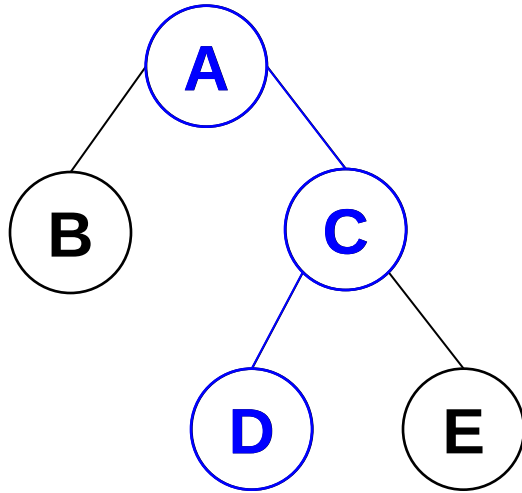
Uma árvore pode ser:

- Vazia ($N=0$)
- Composta por um único nó (raiz)
- Composta por uma raiz e subárvores

Árvores: Caminho

- Um caminho de n_i a n_k , onde n_i é antecedente a n_k , **é a sequencia de nós para se chegar de n_i a n_k .**
- Se n_i é antecedente a n_k , **n_k é descendente de n_i**
- Comprimento = quantidade de arestas do caminho (desconsiderar a raiz).

Árvore: Caminho



Floresta $T = T_A \cup T_F \cup T_I$

– $T_A = \{A, B, C, D, E\}$

– $T_F = \{F, G, H\}$

– $T_I = \{I\}$

- Caminhos:

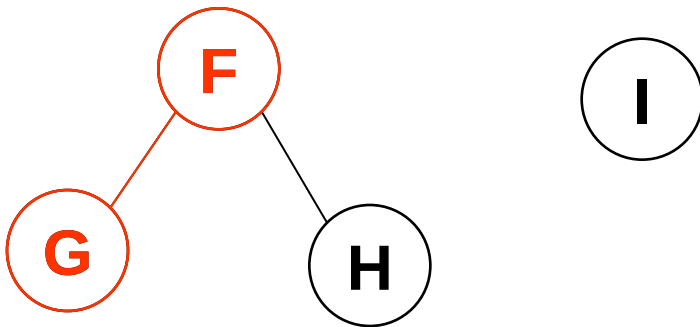
- Caminho1 = (A, C, D)

- Caminho2 = (F, G)

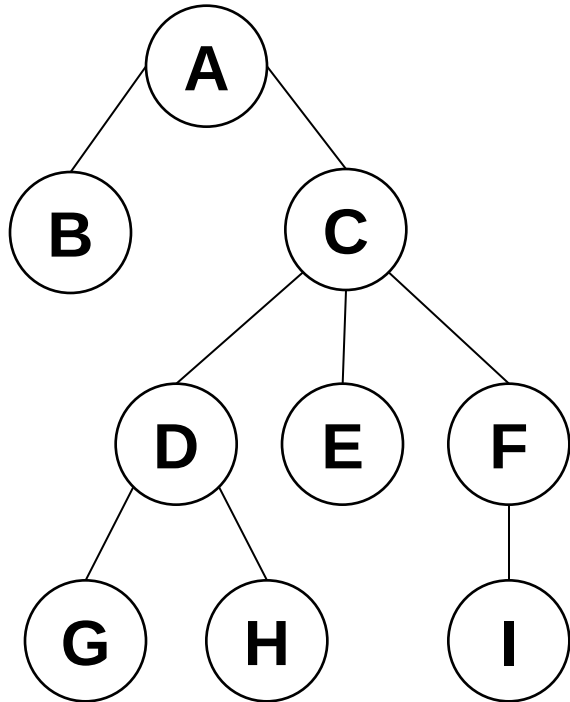
- Comprimentos:

- Comprimento (Caminho1) = 2

- Comprimento (Caminho2) = 1



Árvore: Altura



NÍVEIS	
A	1
B, C	2
D, E, F	3
G, H, I	4

ALTURAS EM RELAÇÃO AO NÓ FOLHA COM MAIOR NÍVEL	
$h(A)$	4
$h(B)$	1
$h(C)$	3
$h(D)$	2
$h(E)$	1
$h(F)$	2
$h(G)$	1
$h(H)$	1
$h(I)$	1

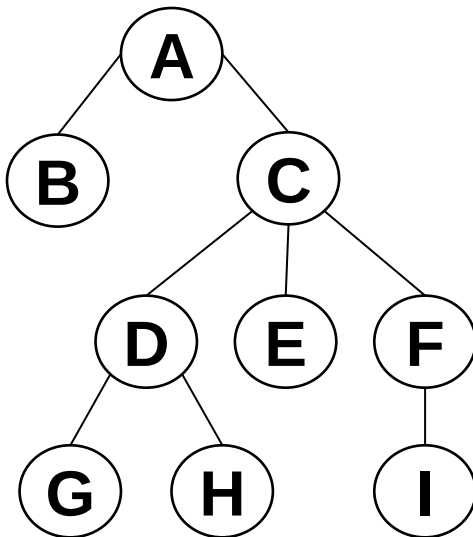
$$h(T) = 4$$



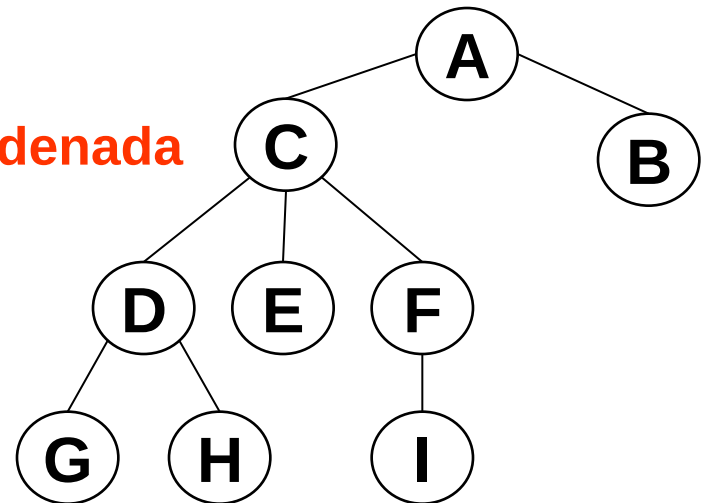
Árvore: Ordenada x Desordenada

Ordenada: Os filhos de cada nó estão ordenados (assume-se ordenação da esquerda para a direita)

Ordenada

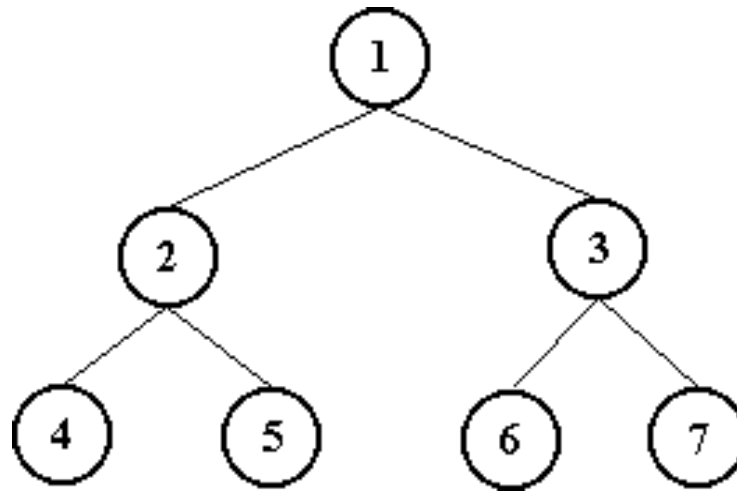


Desordenada



Árvore Cheia

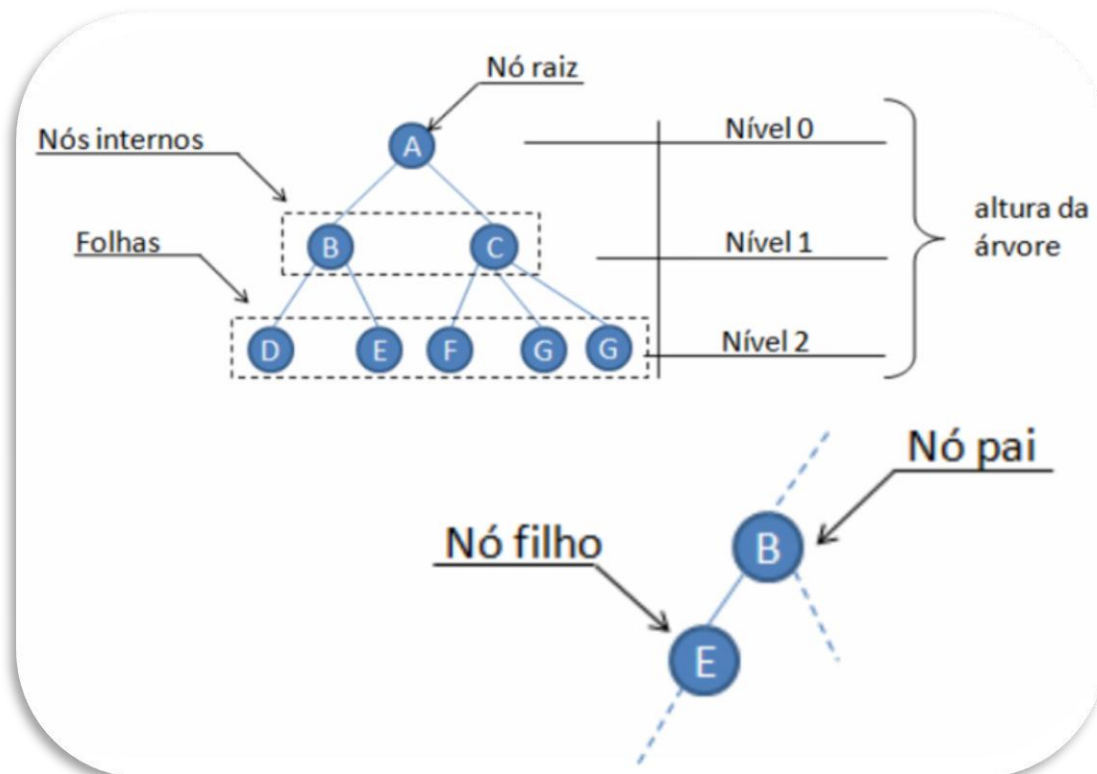
Uma árvore é cheia se possui o número máximo de nós, isto é, todos os nós tem número máximo de filhos exceto as folhas, e todas as folhas estão na mesma altura.



Árvore cheia
de grau 2

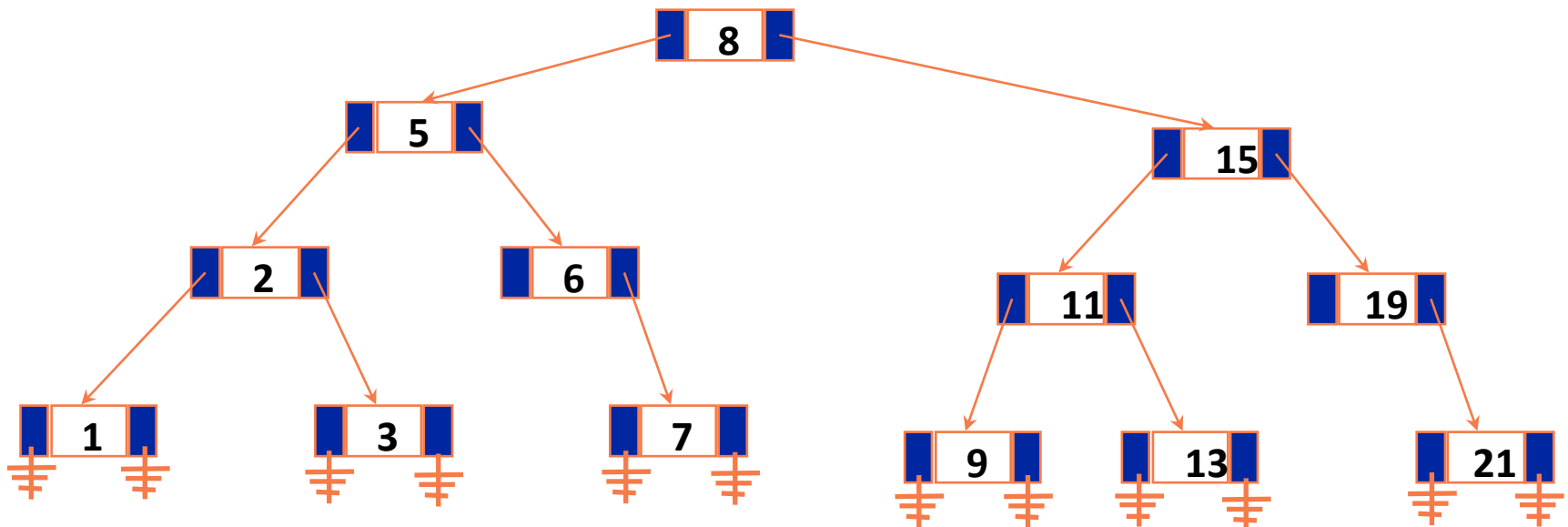
Classificação de Árvores

- Árvores Binárias de Busca
- Árvores Red Black
- Árvores AVL
- Árvores B



Árvores Binárias de Busca

Uma árvore que se encontra organizada de tal forma que, para cada nó t_i , **todas os valores da subárvore à esquerda de t_i são menores que (ou iguais a) t_i e à direita são maiores que t_i .**



Árvores Binárias de Busca

Em uma árvore binária de busca é possível encontrar-se qualquer chave existente descendo-se pela árvore:

- Sempre à esquerda toda vez que a chave procurada for menor do que a chave do nodo visitado;
- Sempre à direita toda vez que for maior.

- A escolha da direção de busca só depende da chave que se procura e da chave que o nodo atual possui.

Observação: A busca de um elemento em uma árvore balanceada com n elementos toma **tempo médio $< \log(n)$, sendo a busca então $O(\log n)$.**

Árvores Binárias de Busca

Problema Deterioração:

- Quando inserimos utilizando a inserção simples, dependendo da distribuição de dados, pode haver deterioração;
- Árvores deterioradas perdem a característica de eficiência de busca.

Ex. 1, 2, 3, 4, 5, 6, 7

<https://www.cs.usfca.edu/~galles/visualization/BST.html>

Árvores AVL

Manter uma **árvore binária de busca balanceada** sob a presença de constantes inserções e deleções é ineficiente.

Para contornar esse problema foi criada a árvore AVL (**Adelson-Velskii e Landis**).

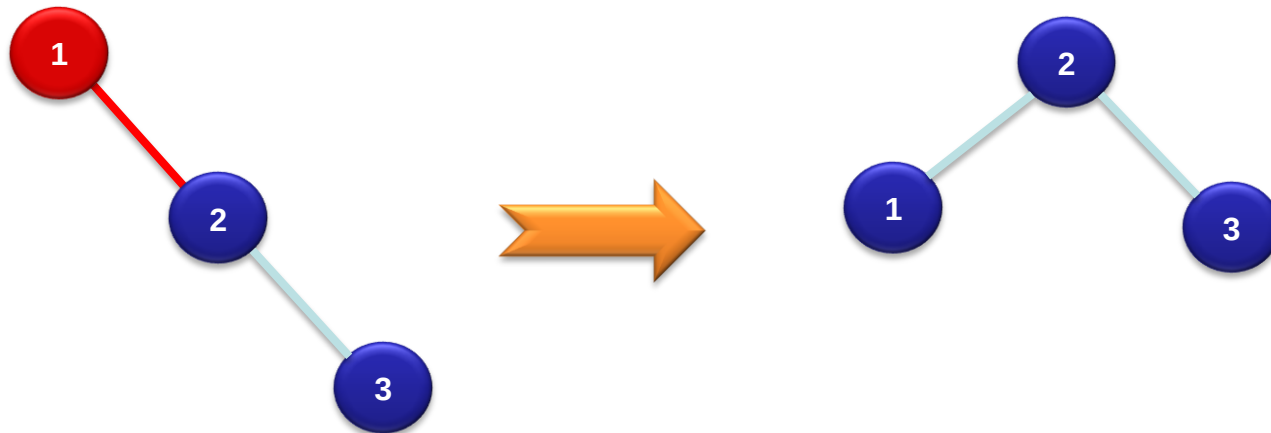
A árvore AVL é uma **árvore binária** com uma **condição de balanço**, porém não completamente balanceada.

Árvores AVL permitem inserção/deleção e rebalanceamento consideravelmente rápidos.

<https://www.cs.usfca.edu/~galles/visualization/AVLtree.html>

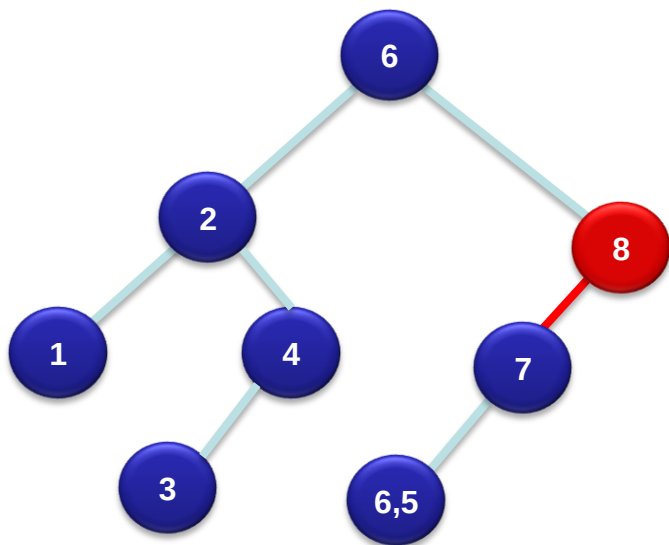
Árvores AVL

A árvore AVL é uma árvore binária com uma **condição de balanço**, porém não completamente balanceada.



Árvores AVL

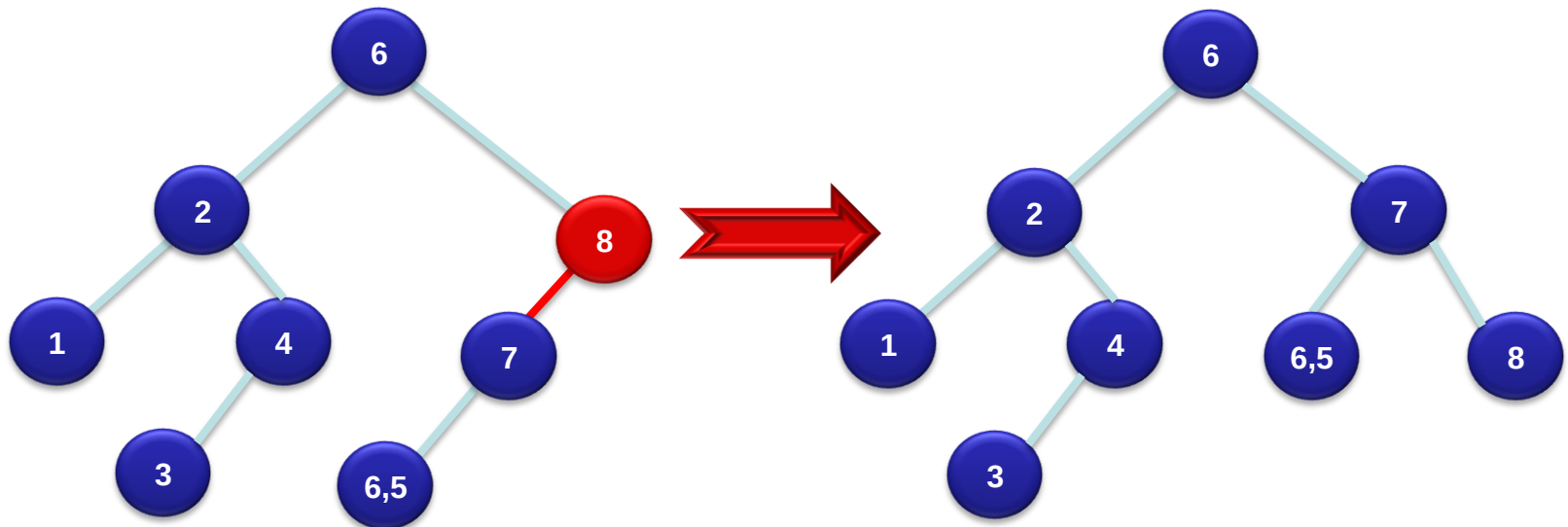
Dado um nodo qualquer, uma árvore está AVL-balanceada se as alturas das **duas subárvores deste nodo diferem de, no máximo, 1** (**Altura Esque. – Alt. Direit. < 2**).



Árvores AVL

Para rebalancear uma árvore após uma inserção, são utilizadas rotações de subárvores:

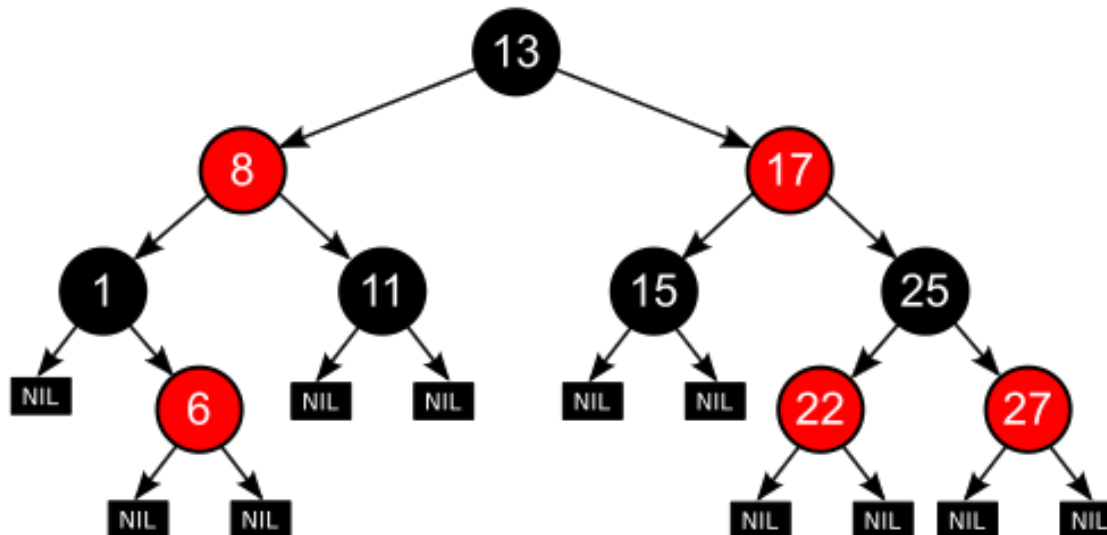
- rotações simples (normalmente).
- rotações duplas (em alguns casos).



Árvores Red Black

Árvores de pesquisa binária com um bit de informação adicional: **a cor**.

- Garante que nenhum caminho é mais que duas vezes mais longo que qualquer outro
- Árvore semibalanceada

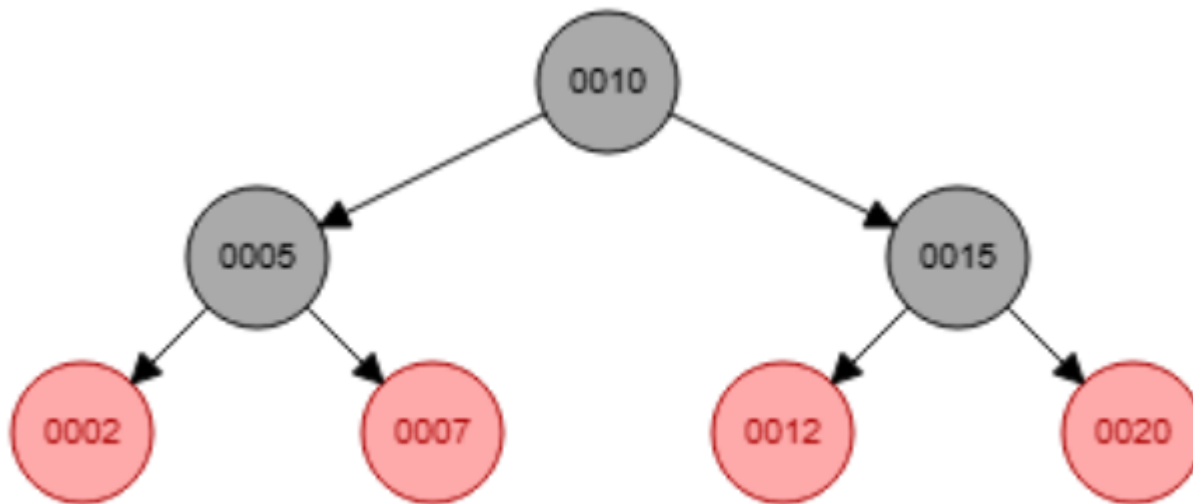


Árvores Red Black

Características:

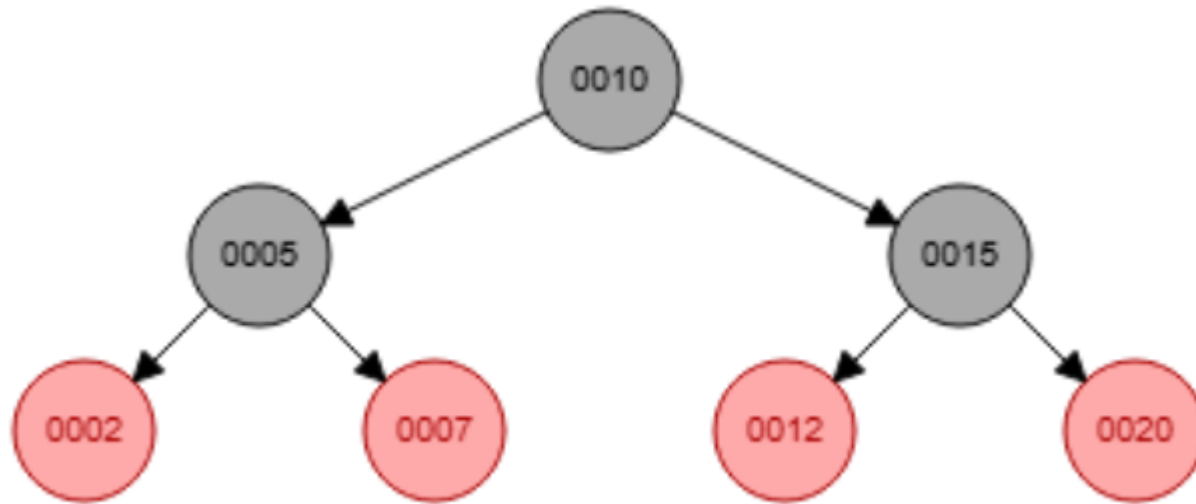
- Todo nodo ou é vermelho ou é negro
- A raiz é negra
- Toda folha é negra
 - **Folhas** são somente os nodos vazios (ponteiros nulos).
- Se um nodo for vermelho, então ambos os filhos são negros.
- Para todo nodo, todos os caminhos até uma folha contém o mesmo número de nodos negros.

Árvores Red Black



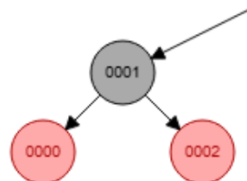
Inserindo o valor 1
Inserindo o valor 0

Árvores Red Black

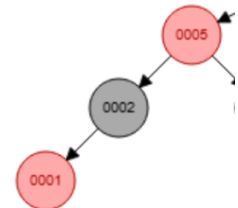


Inserindo o valor 1
Inserindo o valor 0

Inserção Sempre: entra como vermelho



Corrige com recoloração rotações



Comparação AVL x Rubro-Negra

- AVL:
 - árvore muito balanceada
- Rubro-Negra:
 - árvore suficientemente balanceada sem gastar muito.



AVL	Rubro-Negra
Mais balanceada	Menos rígida
Busca mais rápida	Inserção mais rápida
Mais rotações	Menos rotações
Mais difícil manter	Mais usada na prática

Árvores B

São árvores de pesquisa balanceadas **especialmente projetadas** para a pesquisa de informação em memória secundárias:

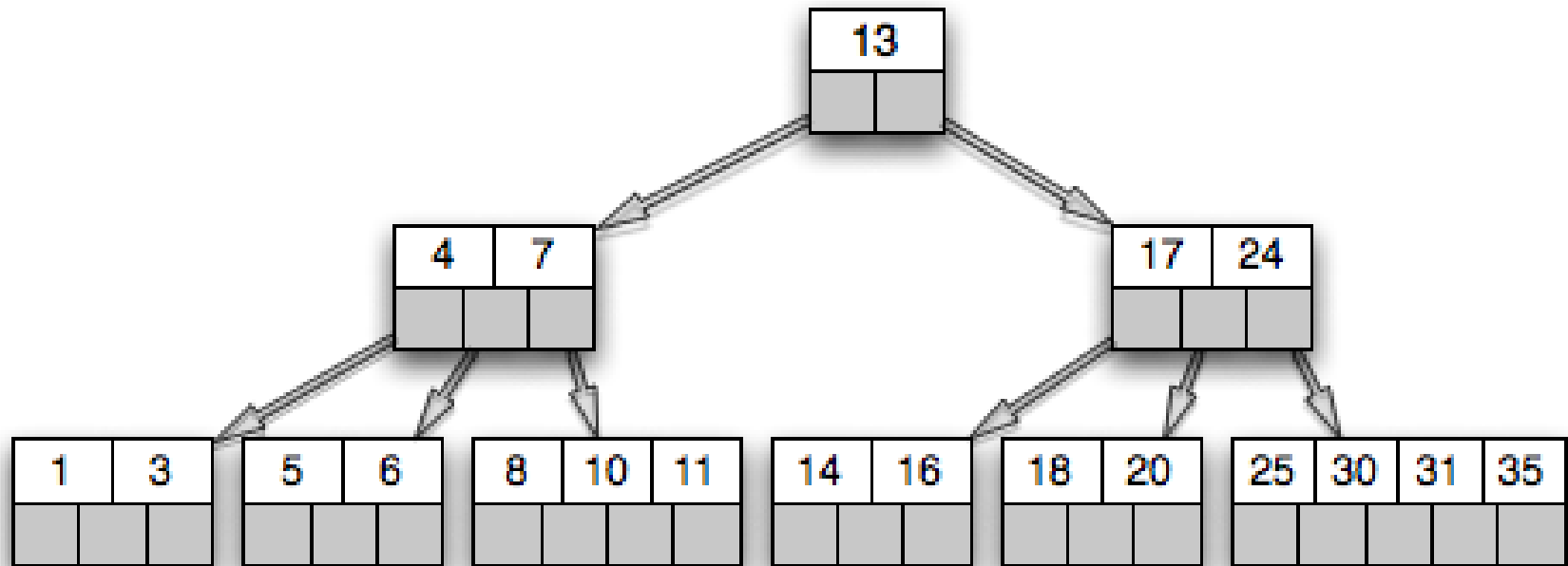
- minimizam o número de operações de movimentação de dados (escrita / leitura) numa pesquisa ou alteração;

Aumentam o número de ramificações na árvore diminuindo, assim, a altura.

Árvores B

Uma **árvore de busca balanceada** com **múltiplos filhos por nó** (não só 2 como em árvores binárias) foi projetada para **minimizar o número de acessos ao disco**, agrupando muitos dados em cada nó.

Árvores B





Implementação de Árvore Binária

Assim como as listas lineares, estruturas de dados do tipo árvore podem ser implementadas:

- Através de arranjos (vetores)
- Através de ponteiros

```
typedef struct Nodo {  
    struct Nodo *esq;  
    char info;  
    struct Nodo *dir;  
};
```

Caminhamento

Diversas formas de percorrer ou caminhar em uma árvore listando seus nós, as principais:

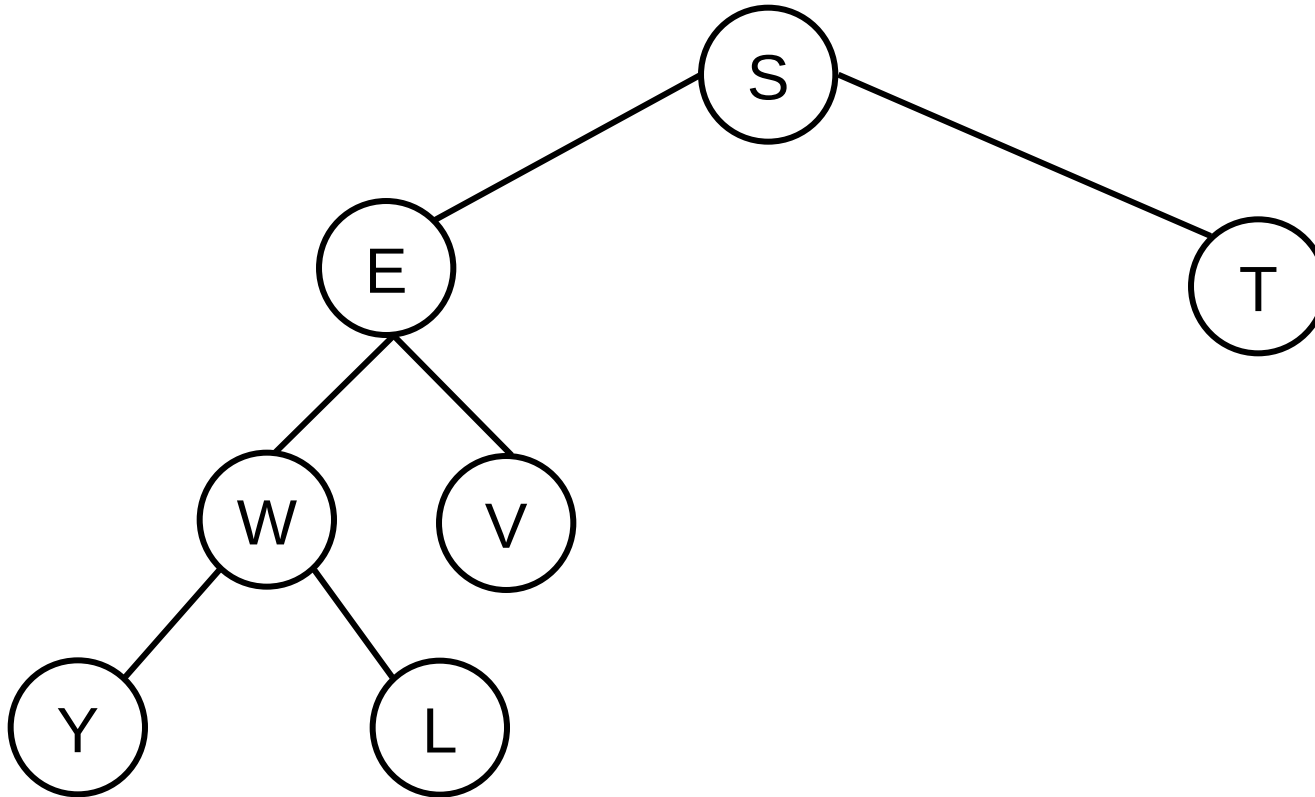
- Pré-ordem (Pré-fixa)
- Central (Infixa)
- Pós-ordem (Pós-fixa)

Pré-ordem

Pré-ordem: lista o nó raiz, seguido de suas subárvores (da esquerda para a direita), cada uma em pré-ordem.

```
void Caminhamento_Pre_Ordem(TArvore *a)
{
    if (!Vazia(a))
    {
        printf("%c ", a->info);
        Caminhamento_Pre_Ordem(a->esq);
        Caminhamento_Pre_Ordem(a->dir); sub_dir
    }
}
```

Pré Ordem: S E W Y L V T

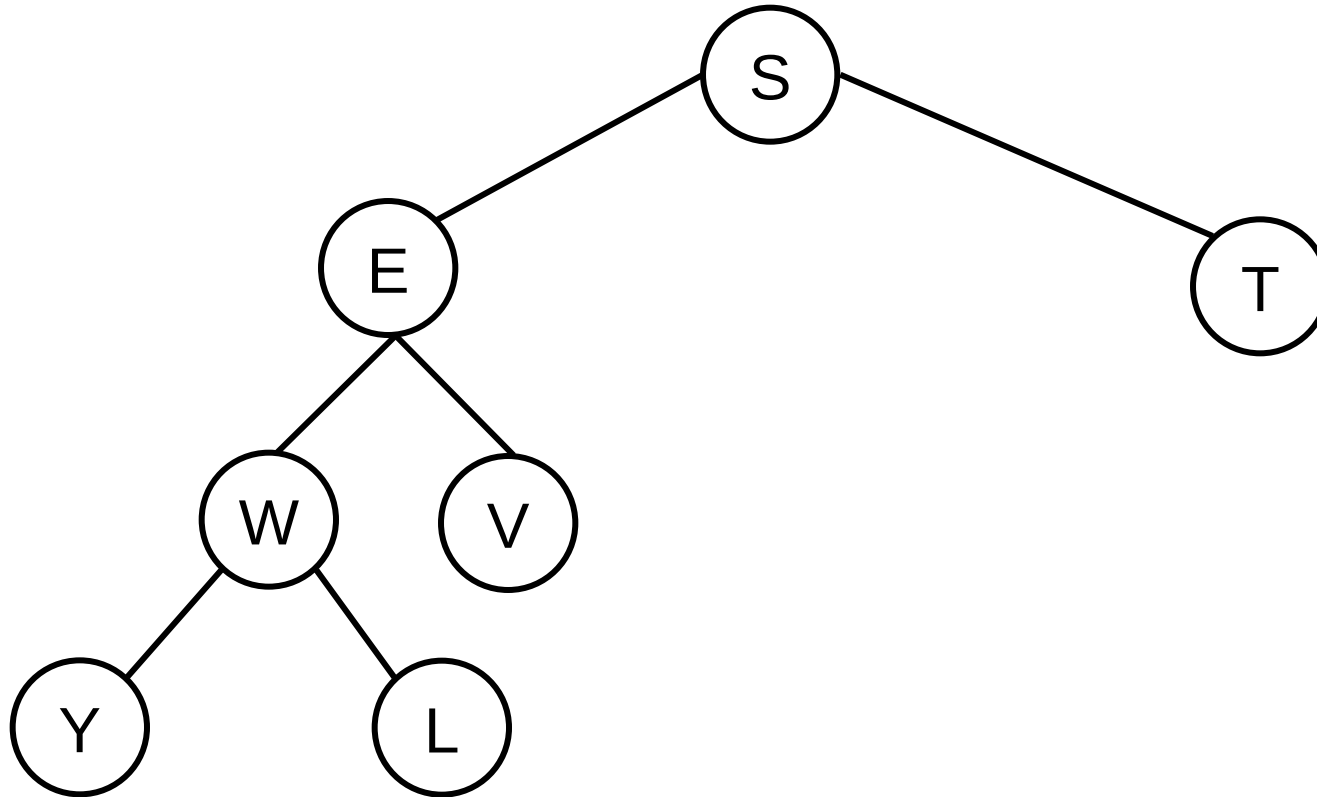


Infixa (Central)

Lista os nós da 1ª. subárvore à esquerda usando o caminhamento central, lista o nó raiz n, lista as demais subárvores (a partir da 2ª.) em caminhamento central (da esquerda para a direita)

```
void Caminhamento_In_Fixado(TArvore *a)
{
    if (!Vazia(a))
    {
        Caminhamento_In_Fixado(a->esq);
        printf("%c ", a->info);
        Caminhamento_In_Fixado(a->dir);
    }
}
```

CENTRAL: Y W L E V S T

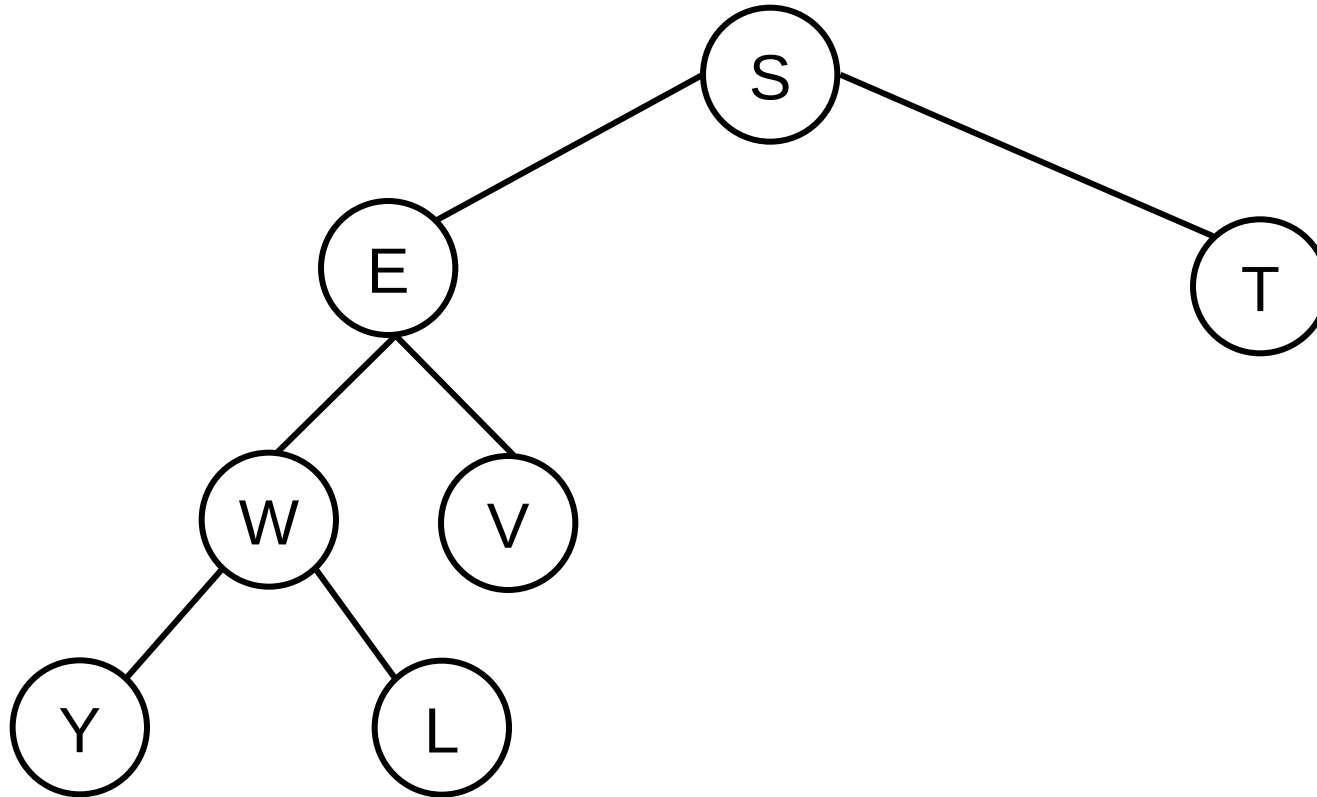


Pós-Ordem

- Lista os nós das subárvores (da esquerda para a direita) cada uma em pós-ordem, lista o nó raiz.

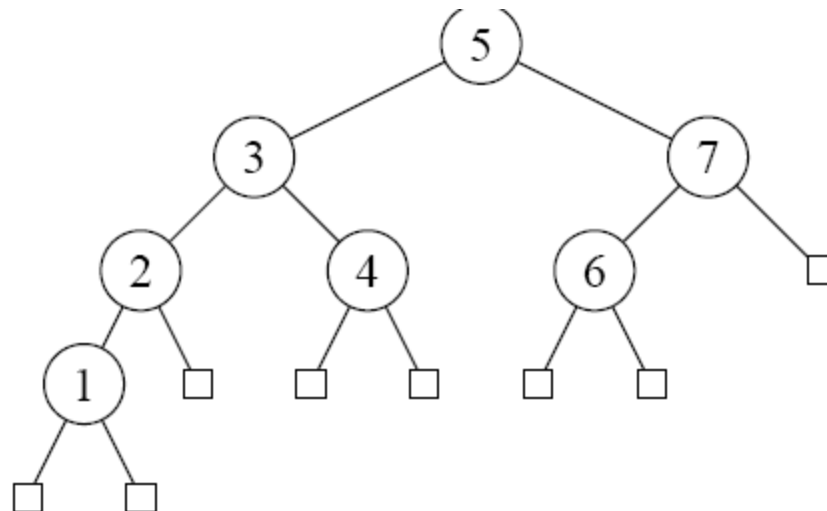
```
void Caminhamento_Pos_Fixado(TArvore *a)
{
    if (!Vazia(a))
    {
        Caminhamento_Pos_Fixado(a->esq);
        Caminhamento_Pos_Fixado(a->dir);
        printf("%c ", a->info);
    }
}
```

PÓS ORDEM: Y L W V E T S



Altura de Árvore

- O nível do nó raiz é 1.
 - Se um nó está no nível i então a raiz de suas subárvores estão no nível $i + 1$.
 - A altura de um nó é o comprimento do caminho mais longo deste nó até um nó folha.
 - A altura de uma árvore é a altura do nó raiz.



Pesquisa

- **Para encontrar um registro com uma chave x :**
 - Compare-a com a chave que está na raiz.
 - Se x é menor, vá para a subárvore esquerda.
 - Se x é maior, vá para a subárvore direita.
 - Repita o processo recursivamente, até que a chave procurada seja encontrada ou um nó folha é atingido.
 - Se a pesquisa tiver sucesso então o conteúdo do registro retorna no próprio registro x .

Inserção

Onde inserir?

- Atingir um apontador nulo em um processo de pesquisa significa uma pesquisa sem sucesso.
- O apontador nulo atingido é o ponto de inserção.

Como inserir?

- Cria célula contendo registro
- Procura lugar na árvore
- Se registro não tiver na árvore, insere-o

```
if (valor < no->info)
    esquerda
else
    direita
```

Comparação de Strings

- O C trata string como um vetor. Ou seja, Não é possível comparar dois vetores com “==”.
- Uma string em C é um vetor de caracteres (char[]), e o operador == compara apenas os endereços de memória.
- Para comparar nomes, utiliza-se a função:
`strcmp()` da biblioteca: `<string.h>`

Comparação de Strings

Sintaxe:

`strcmp(string1, string2)`

- A comparação é realizada caractere por caractere.

Retornos possíveis

- 0 para strings iguais
- < 0 para a primeira string antes (alfabeticamente)
- > 0 para a primeira string vem depois (alfabeticamente)

Comparação de Strings

```
1  #include <stdio.h>
2  #include <string.h>
3
4  int main() {
5
6      char string1[10];
7      char string2[10];
8
9      printf("Digite uma string: ");
10     scanf("%s", string1);
11
12     printf("Digite outra string: ");
13     scanf("%s", string2);
14
15     if (strcmp(string1, string2) == 0) {
16         printf("strings iguais\n");
17     } else if (strcmp(string1, string2) < 0) {
18         printf("String1 antes de string2\n");
19     } else if (strcmp(string1, string2) > 0) {
20         printf("String2 antes de string1\n");
21     }
22
23     return 0;
24 }
```

Comparação de Strings

Dado o exemplo de código anterior, qual será a saída para as entradas “a” e “A”?

Dado o exemplo de código anterior, qual será a saída para as entradas “Aa” e “aA”?

Dado o exemplo de código anterior, qual será a saída para as entradas “Aac” e “AAb”?

Algoritmo de Remoção

Alguns comentários:

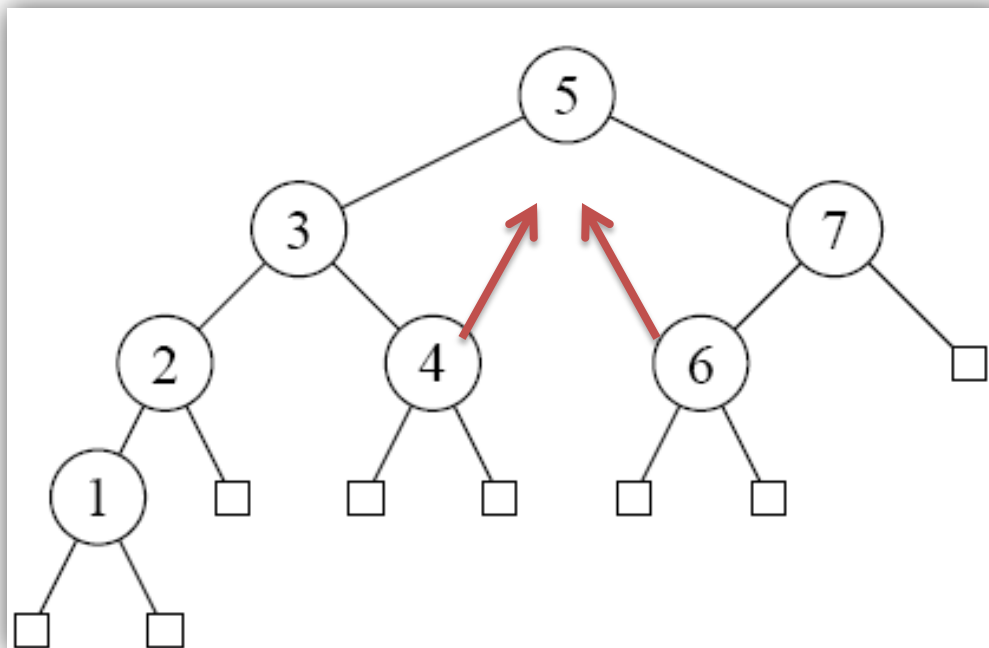
- A retirada de um registro não é tão simples quanto a inserção.
- Se o nó que contém o registro a ser retirado possui no máximo um descendente $\Rightarrow$ **a operação é simples.**
- No caso do nó conter **dois descendentes** o registro a ser retirado deve ser primeiro substituído pelo registro mais à direita na subárvore esquerda ou pelo registro mais à esquerda na subárvore direita.

Algoritmo de Remoção

- **Nó folha (sem filhos)**
 - Basta remover o nó.
- **Nó com apenas 1 filho**
 - O nó pai do nó passa a apontar diretamente para o filho.
- **Nó com 2 filhos**
 - Necessário substituir o nó pelo **maior valor da subárvore esquerda OU o menor valor da subárvore direita.**

Algoritmo de Remoção

Ou seja, para retirar o registro com chave 5 da árvore basta trocá-lo pelo registro com chave 4 ou pelo registro com chave 6, e então retirar o nó que recebeu o registro com chave 5.



Algoritmo de Remoção

```
90 □ TARvore* remove_binario(TARvore* a, int info) {
91     if (a == NULL) return NULL;
92
93 □   if (info < a->info) {
94       a->esq = remove_binario(a->esq, info);
95   } else if (info > a->info) {
96       a->dir = remove_binario(a->dir, info);
97   } else {
98       // Encontrou o nó a remover
99 □   if (a->esq == NULL && a->dir == NULL) {
100       free(a);
101       return NULL;
102   } else if (a->esq == NULL) {
103       TARvore* temp = a->dir;
104       free(a);
105       return temp;
106   } else if (a->dir == NULL) {
107       TARvore* temp = a->esq;
108       free(a);
109       return temp;
110   } else {
111       // Dois filhos: encontrar o menor da subárvore direita
112       TARvore* temp = a->dir;
113       while (temp->esq != NULL)
114           temp = temp->esq;
115       a->info = temp->info;
116       a->dir = remove_binario(a->dir, temp->info);
117   }
118 }
119 return a;
120 }
```

Árvores Balanceadas

- A vantagem de uma árvore balanceada com relação a uma degenerada está em sua eficiência.
 - Por exemplo: numa árvore binária degenerada de 10.000 nós são necessárias, em média, 5.000 comparações (semelhança com arrays ordenados e listas encadeadas).
- Numa árvore balanceada com o mesmo número de nós essa média reduz-se a **14 comparações**.

Árvores Balanceadas

- Inicialmente inserimos um novo nó na árvore normalmente.
- A inserção deste pode degenerar a árvore.
- A restauração do balanceamento é feita através de rotações na árvore no nó “pivô”.
- Nó “pivô” é aquele que após a inserção possui Fator de Balanceamento fora do intervalo.

Árvores de Balanceadas

- Rotação simples para a direita / Esquerda

